

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Report

Topic 4 – Async Research Assistant

Team:	Binary	Date:	July 27, 2026
Repo:	https://github.com/mrorxan/research-assistant	Tag:	v1.0-final
Members:	Orkhan Aliyev (@mrorxan), Habil Huseynov (@Habil7)		

Executive Summary

We built an async research assistant: the user asks a question, the system fetches excerpts from Wikipedia, arXiv, and a Tavily web search concurrently, and a Gemini model writes a short answer in which every claim carries a bracketed citation back to a source. The headline concurrency number is a **2.99×** speedup over a sequential baseline on a deterministic three-source workload, effectively the theoretical maximum of $3\times$. The robustness story we are most confident in happened live rather than in a test: during a demo run Gemini returned HTTP 503 twice in a row, our tenacity policy backed off 1s and then 2s, and the third attempt succeeded without the user seeing anything but a slightly slower answer. The most surprising lesson was how hostile the real world is to naive HTTP clients: Wikipedia rejects requests without a policy-compliant User-Agent, Windows consoles crash on characters the LLM likes to emit, and an LLM provider can be down for exactly the two seconds you call it. Most of our code exists to absorb precisely such facts.

Contents

Executive Summary	1
1 Project Overview	1
1.1 Problem framing & scope	1
1.2 Provider choice and the AI module contract	2
1.3 Sample run	2
2 Architecture	2
2.1 Module map	3
2.2 OOP design & key abstractions	3
2.3 Data flow across module boundaries	4
3 Concurrency & Performance	4
3.1 Why this concurrency model	4
3.2 Sequential-vs-concurrent benchmark	4
3.3 Rate limit & politeness	5
4 Robustness & Error Handling	5
4.1 Wrapping the AI module	5
4.2 Failure-mode analysis (one concrete incident)	5
4.3 Input validation	6

5	Storage & Caching	6
5.1	Schema	6
5.2	Caching policy	6
6	Testing	6
6.1	Strategy & coverage	6
6.2	Notable tests	7
7	Deployment & Reproducibility	7
7.1	Docker image	7
7.2	Configuration	8
8	Bonus Features	8
9	Limitations & Future Work	8
10	Tools & Acknowledgements	9
	References	9
A	Appendix — Reproducing the benchmark	9

1. Project Overview

1.1 Problem framing & scope

The system answers a research question by aggregating short excerpts from three independent sources, Wikipedia, arXiv, and a web search backend, and asking a large language model to write a 3–6 sentence answer that cites those excerpts with [N] markers. Input is a natural-language question of at most 500 characters; output is a structured `AnswerWithCitations` with the answer text and the citation-to-source mapping. The only entry point is the CLI, `python -m researcher ask "..."`; Topic 4 does not require an HTTP server and we did not build one.

Our provider stack is Gemini 2.5 Flash Lite for synthesis (free tier on Google AI Studio) and Tavily for web search (free tier, 1000 requests per month). Wikipedia and arXiv need no keys. The provider-agnostic factory inside the provided `ai/` package means this is configuration, not code: switching to Anthropic or OpenAI is an environment-variable change.

We tightened the `TOPIC.md` spec in three places. First, a `clear-cache` CLI command in addition to the required `-no-cache` flag, because deleting stale cache entries by hand during development was error-prone. Second, a timeout on the LLM synthesis call, not only on the source fetches; without it a hung provider would hang the CLI forever. Third, empty fetch results are never cached, so a source that failed quietly (or a question that simply matches no Wikipedia title) is retried on the next ask instead of serving nothing for a whole TTL. We loosened the spec by choosing filesystem JSON over PostgreSQL for the cache, which `TOPIC.md` explicitly permits.

1.2 Provider choice and the AI module contract

We call exactly four functions from the provided package: `ai.fetch_wikipedia`, `ai.fetch_arxiv`, `ai.fetch_web`, and `ai.synthesize`. **We did not modify the public interface of the provided `ai/` package**; the copied files are byte-identical to the distributed ones, and all 16 provided smoke tests pass unchanged both locally and inside the Docker image.

Defence against semantically wrong model output is layered. The provided synthesizer already drops citation indices that point outside the source list, so a hallucinated [99] never reaches the user.

Above that, our `ResearchSession` model is declared with `extra="forbid"`, so an unexpected field crossing an SE-layer boundary raises a `ValidationError` instead of propagating silently. An answer with zero citations is still returned and rendered; the CLI simply prints no references block.

1.3 Sample run

A real first run, July 20, 2026, abbreviated to the interesting parts:

```
$ python -m researcher ask "What is photosynthesis and what are its main stages?"

Q: What is photosynthesis and what are its main stages?

A: Photosynthesis is a system of biological processes where photopigment-bearing organisms convert light energy into chemical energy to fuel their metabolism [1]. [...] The main stages are the light-dependent reactions and the Calvin cycle [5, 6].

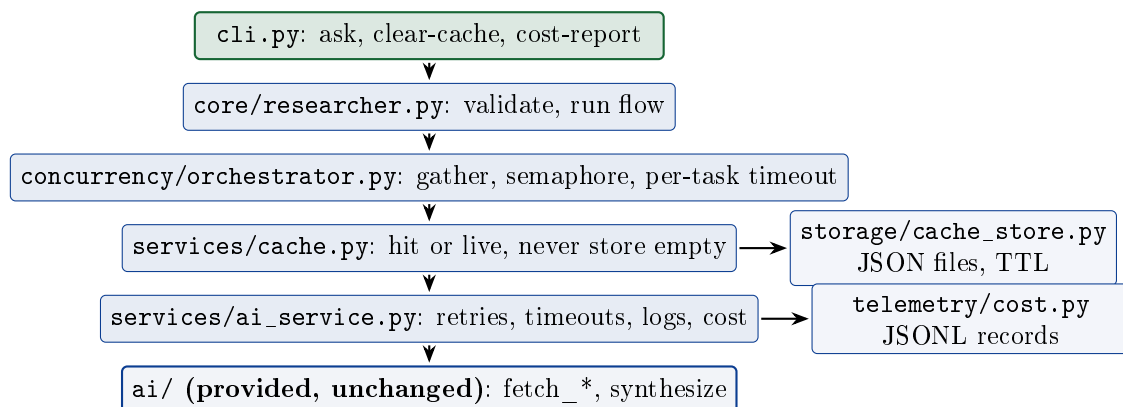
References:
[1] (wikipedia) Photosynthesis
    https://en.wikipedia.org/wiki/Photosynthesis
[5] (web) Photosynthesis
    https://education.nationalgeographic.org/resource/photosynthesis
[6] (web) The process of photosynthesis - Student Academic Success
    https://www.monash.edu/student-academic-success/biology/...

timings: 9210 ms | cache: wikipedia=miss, arxiv=miss, web=miss
```

The 9.2s includes two live Gemini 503 responses and the retry back-off described in §4. Asking the same question again took 4.8s with `arxiv=hit`, `web=hit`; the scripted demo over all five sample questions (`scripts/demo.py`) measured 3.6–5.1s per question with a warm connection pool and no retries, and its full output is committed under `artefacts/demo_run.json`.

2. Architecture

2.1 Module map



Exactly one module imports the `ai` package: `services/ai_service.py`. Four call sites cross that boundary (the three fetchers plus `synthesize`). Exactly one module crosses the storage boundary: the cache facade, through `CacheStore.get` and `put`. Swapping the LLM provider changes one environment variable and zero source files. The synthesis path bypasses the cache and the orchestrator by design: `Researcher` calls `AIService.synthesize` directly, because an LLM answer depends on the fetched sources and is not a cacheable, per-source artefact.

Module-by-module summary:

- `config.py` – pydantic-settings model; 17 typed fields with validators, cached by `lru_cache`.
- `models.py` – `ResearchSession`: question, sources, answer, duration, per-source cache hits, failed sources.
- `storage/cache_store.py` – one JSON file per (source, canonicalised query), TTL-aware, corruption-tolerant.
- `telemetry/cost.py` – appends one JSONL record per LLM call; summarised by `cost-report`.
- `services/ai_service.py` – tenacity retries, the synthesize thread-plus-timeout, structured logs, cost recording.
- `services/cache.py` – cache facade returning `FetchResult(sources, cache_hit)`; never stores empty lists.
- `services/failover.py` – `FailoverLLM`, an `LLMProvider` that tries a configured chain of providers.
- `concurrency/orchestrator.py` – the parallel fan-out with per-task isolation.
- `core/researcher.py` – validate, fetch, synthesise, package.
- `cli.py` – click commands, UTF-8 console setup, output rendering.

2.2 OOP design & key abstractions

The provided `ai/providers/base.py` defines the abstract `LLMProvider` with three concrete subclasses selected by a factory. Our own use of inheritance plugs into exactly that seam: `FailoverLLM` subclasses `LLMProvider`, which is what lets `ai.synthesize(llm=...)` accept a failover chain without the provided package knowing failover exists.

```

1 class FailoverLLM(LLMProvider):
2     """Tries each configured provider in order until one answers."""
3
4     def __init__(self, factories: Sequence[tuple[str, ProviderFactory]]) -> None:
5         if not factories:
6             raise ValueError("at least one provider factory is required")
7         self._factories = list(factories)
8         self._instances: dict[str, LLMProvider] = {}

```

Providers are built lazily from factories, so the fallback SDK and API key are only needed if the primary actually fails. Everywhere else we use composition: `CachedAIService` holds an `AIService` and a `CacheStore`; the orchestrator holds the cached service; `Researcher` holds the orchestrator. If asked why these are not abstract base classes: each of those roles has exactly one implementation at runtime, so an ABC would add indirection without reuse. The failover chain is the one place where polymorphism earns its keep, and there we inherit.

2.3 Data flow across module boundaries

Typed values cross every boundary: `Settings` (configuration), `Source` and `AnswerWithCitations` (reused from `ai/`), `FetchResult` and `SourceResult` and `FetchOutcome` (frozen dataclasses inside the fetch path), and `ResearchSession` (returned to the CLI). No naked dictionaries flow between modules; the only dicts in play are the `cache_hits` map inside `ResearchSession` and the JSON payloads at the storage boundary, where JSON is the point. We wrapped the provided models in `ResearchSession` exactly when SE-layer concerns appeared (timing, cache state, failure list) that do not belong to the AI module.

One deliberate detail: cache-hit information travels in return values rather than in shared mutable state on the service object, so two concurrent asks against the same service cannot interleave their bookkeeping.

3. Concurrency & Performance

3.1 Why this concurrency model

The workload is three I/O-bound HTTP calls per question, each dominated by network round-trips. `asyncio` with `asyncio.gather` runs them on one event loop with negligible overhead, and the provided fetchers are already coroutines, so the async model is native rather than bolted on. Threads would add context-switch cost for no parallel gain under the GIL; process pools are for CPU-bound work, which this is not. Parallelism is bounded by an `asyncio.Semaphore` configured through `MAX_PARALLEL` (default 3, one slot per source). At 1 the system degenerates to sequential fetching; far above 3 the bound stops mattering for a single ask but protects politeness if a batch script funnels many questions through one orchestrator.

3.2 Sequential-vs-concurrent benchmark

The benchmark replaces the fetchers with stubs that sleep 0.5s each, so the measurement is deterministic and isolates the orchestrator’s own overhead from network variance. Command and result (Windows 11 laptop, Python 3.12.7, dependencies from the pinned `requirements.txt`):

```
python scripts/bench.py --N 5 --max-parallel 3
```

Workload	<i>N</i>	Sequential (s)	Concurrent (s)	Speedup
5 iterations × 3 sources, 0.5 s each	5	7.589	2.535	2.99×

Three equally slow sources cap the theoretical speedup at 3×; measuring 2.99× says the gather fan-out and semaphore add no observable overhead at this scale. The honest caveat is that live speedup is smaller because the sources are not equally slow and the LLM call is serial: in the live demo the end-to-end time per question was 3.6–5.1 s, of which roughly half is synthesis. After parallelising the fetches, the bottleneck is $\max(t_{\text{wiki}}, t_{\text{arxiv}}, t_{\text{web}})$ plus the LLM; the next real win would be streaming the LLM output, not more fetch parallelism.

3.3 Rate limit & politeness

Request-count concurrency is bounded by the semaphore; we did not add token-bucket limiting because the free tiers involved (Wikipedia, arXiv, Tavily, Gemini) are generous at our scale and the retry policy absorbs transient 429/503 responses. This is not hypothetical: Gemini’s free tier returned real 503s during development, and the exponential backoff (1s, 2s, capped at 8s, three attempts) recovered every time. The highest burst our code produces is three concurrent fetches per ask. One politeness detail proved mandatory rather than optional: Wikipedia rejects API requests whose User-Agent does not identify the client and a contact, returning HTTP 403. We verified this empirically (the default client and a browser-imitating string are both rejected; a policy-compliant string is accepted) and set a compliant User-Agent on the single shared `httpx.AsyncClient` that all fetchers reuse.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Every `ai.*` call goes through `AIService`. Retries: `tenacity`, `retry_if_exception_type(ProviderError)`, three attempts, exponential backoff between 1s and 8s, with a `WARNING` log before each sleep.

Anything that is not a `ProviderError`, such as a `ValueError` from bad input, is a programmer error and is deliberately not retried; a unit test pins this distinction. Timeouts exist at two levels: each source fetch runs inside its own `asyncio.timeout` window (default 15s) owned by the orchestrator, and the blocking `ai.synthesize` call runs in a worker thread bounded by `asyncio.wait_for` (default 60s). On synthesis timeout the caller gets a clean `ProviderError`; the abandoned worker thread finishes in the background, an accepted trade-off for a short-lived CLI process, and the docstring says so.

Logging is structured key=value text: `fetch_start/fetch_done` with source and result count at INFO, `source_timeout/source_failed` and retry notices at WARNING. No API key or payload is ever logged.

4.2 Failure-mode analysis (one concrete incident)

The incident we designed for arrived on its own. During a live run on July 20, Gemini returned HTTP 503 ("This model is currently experiencing high demand") on the first synthesis attempt, again on the second, and succeeded on the third. The log shows the policy working exactly as written:

```
13:36:49 WARNING Retrying in 1 seconds as it raised ProviderError:
          Gemini call failed: 503 UNAVAILABLE ...
13:36:52 WARNING Retrying in 2 seconds as it raised ProviderError:
          Gemini call failed: 503 UNAVAILABLE ...
13:36:56 INFO     synthesize_done n_citations=2
```

The user saw a 13s answer instead of a crash. The same class of failure is pinned by unit tests on the fetch path: `test_one_source_timeout_does_not_block_others` injects a 1s delay into a fake Wikipedia fetcher under a 0.05s timeout and asserts that arXiv and web still deliver, with `failed == ["wikipedia"]` surfaced to the user as a note under the answer. Writing that test drove the design: the first sketch let the timeout exception propagate through `gather` and cancel the batch; converting failures into structured `SourceResult` values is what made per-source isolation real.

A second real-world failure worth recording: on Windows the default console encoding (cp1252) crashed the first time Gemini emitted a Unicode arrow in an answer. The CLI now reconfigures stdout and stderr to UTF-8 before printing anything.

4.3 Input validation

- **CLI.** Empty or whitespace-only questions raise `QuestionValidationError`, exit code 2. Questions over `MAX_QUESTION_LENGTH` (default 500) likewise. A 50 000-character question is rejected before any network call. `-sources` values outside {wiki, wikipedia, arxiv, web} raise a `click.UsageError`.
- **Environment.** `Settings` validators reject an unknown `LOG_LEVEL`, non-positive integers (`MAX_PARALLEL=0`), and non-positive timeouts at startup, naming the offending field.
- **Cache files.** A corrupted JSON cache entry is logged at WARNING and treated as a miss, never a crash. Cache filenames are derived from a SHA-256 digest of the canonicalised query, so user input cannot traverse the cache directory.
- **Model output.** Out-of-range citation indices are dropped by the provided synthesizer; unexpected fields on our own models are rejected by `extra="forbid"`.

5. Storage & Caching

5.1 Schema

There is no relational schema; the cache is a flat directory of JSON files, one per (source, query) pair, named `{source}_{sha256[:24]}.json` where the digest covers `source::canonicalised_query`. Each file holds:

```
{
  "stored_at": 1784543740.29,      # POSIX timestamp at write time
  "value": [                       # list[Source.model_dump()]
    {"title": "...", "url": "...", "snippet": "...", "origin": "arxiv"}
  ]
}
```

The live provider is the source of truth; the cache is a derived view invalidated purely by age (`CACHE_TTL_SECONDS`, default 24h). If a future `ai/` upgrade changed the `Source` schema, re-validation through `Source(**item)` on read would fail loudly and the entry would be refetched. Cost telemetry uses the same directory: an append-only `costs.jsonl`, one record per LLM call.

5.2 Caching policy

We cache the per-source `list[Source]`, keyed by canonicalised query (lower-cased, trimmed, whitespace-collapsed), so "What is X?" and "what is x?" share an entry. `-no-cache` bypasses both read and write. Two policy decisions matter. First, *empty results are never cached*: an empty list usually means the source had a bad moment, or the question does not match a Wikipedia title, and remembering that for 24 hours would keep serving nothing after the source recovers. Second, the hit/miss state of every source is returned to the user on every ask (`cache: wikipedia=miss, arxiv=hit, ...`), which turned out to be the single most useful debugging signal in development. Measured behaviour on a repeated ask: arXiv and web hit; Wikipedia misses again exactly when its first response was empty, which is the no-negative-caching rule doing its job.

6. Testing

6.1 Strategy & coverage

The suite contains **92 tests**: 76 of ours plus the 16 provided smoke tests, running in about 6 seconds, entirely offline. We mock at the `ai.*` function boundary (monkeypatching `researcher.services.ai_service.fetch_wikipedia` and friends) rather than inside the provided package, so the smoke tests keep exercising the real `ai/` code. The provided smoke tests pass unmodified; the file is byte-identical to the distributed one and is excluded from our linter so no tool can silently rewrite it.

Check	Result
pytest (76 own + 16 provided smoke)	92 passed
Coverage on <code>researcher/</code> (pytest -cov)	96 %
mypy (17 source files, <code>disallow_untyped_defs</code>)	0 issues
ruff (lint + import order, rules E, F, W, I, UP, B)	0 findings

Per module, coverage is 100 % on the orchestrator, config, core flow, models, and cache facade; the uncovered 4 % sits in defensive branches (cache-file write errors, retry-loop fallbacks) and the CLI's live wiring function, which is exercised by the real demo rather than by mocks.

6.2 Notable tests

- **Happy path, end to end.** `test_full_flow_with_mocked_ai` patches all four `ai.*` functions, runs a real `Researcher` built by the CLI wiring, and asserts sources, answer, and empty failure

list.

- **Concurrency degradation.** `test_one_source_exception_does_not_kill_others` makes the web fetcher raise; the other two sources still deliver and `failed == ["web"]`. A companion test asserts the three fetches actually overlap in time, not just complete.
- **Error-path retry discrimination.** `test_fetch_wikipedia_retries_then_succeeds` counts calls to prove exactly one retry happened; `test_value_error_is_not_retried` proves programmer errors are not retried. Together they pin the retry predicate, which is where a wrong exception class would silently destroy the policy.
- **Negative-caching rule.** `test_empty_results_are_not_cached` feeds an empty result, then a non-empty one, and asserts the second call went live.
- **Failover chaos test.** `test_fallback_used_when_primary_call_fails` kills the primary provider and asserts the answer comes from the secondary; a lazy-construction test asserts the fallback SDK is never touched while the primary is healthy.

What we did not test: live provider behaviour inside the suite, deliberately, per the rubric. The live path is exercised by `scripts/demo.py`, whose committed output is the evidence.

7. Deployment & Reproducibility

7.1 Docker image

Multi-stage build on `python:3.12.7-slim`: the build stage installs the pinned requirements into `/opt/venv` and strips pip; the runtime stage copies only that venv plus the source, and runs as a non-root user.

```
docker build -t research-assistant .
docker run --rm --env-file .env research-assistant
docker run --rm research-assistant python -m pytest -q    # 92 passed in-container
```

We verified all three commands on the submission machine: the default command runs a full live ask (fetch, synthesis, citations) inside the container, and the entire test suite passes in-container. Compressed image content is 86 MB; the unpacked dependency venv is 153 MB, dominated by numpy (70 MB), which the provided `ai/` package imports and we therefore cannot drop. Python inside the image (3.12.7) matches the development venv exactly, which closes the class of works-on-my-machine bugs where code is written on one interpreter and shipped on another.

7.2 Configuration

All configuration is environment-driven and documented in `.env.example` and the README table: provider selection (`LLM_PROVIDER`, `LLM_MODEL`, keys), optional failover (`LLM_FALLBACK_PROVIDER`, `LLM_FALLBACK_MODEL`), web search (`WEB_SEARCH_PROVIDER`, `TAVILY_API_KEY`), and the runtime knobs (log level, cache location and TTL, per-source and synthesis timeouts, source and parallelism limits, question length cap, retry policy). Invalid values fail at startup with the field name; missing API keys surface at the first provider call as a `ProviderError` with a clear message. `.env` is gitignored and was never committed; `.env.example` carries every variable with its default.

8. Bonus Features

- **Multi-provider failover.** `FailoverLLM` implements the provided `LLMProvider` ABC and is handed to `ai.synthesize(llm=...)` when `LLM_FALLBACK_PROVIDER` is set. Providers are constructed lazily; chaos tests kill the primary at call time and at construction time (missing

API key) and assert the secondary answers. Disabled by default so the core path stays identical to a no-bonus project.

- **Cost telemetry.** Every synthesis call appends provider, model, estimated prompt and completion tokens, and an estimated dollar cost to `costs.jsonl`; `python -m researcher cost-report -since N` summarises totals and a per-model breakdown. Because the provided interface returns plain text, tokens are estimated as characters divided by four, and the report says so. Over development we recorded 14 synthesis calls, roughly 14.8k prompt and 2.1k completion tokens, \$0.0023 in total, about \$0.00017 per question.
- **GitHub Actions CI.** On every pull request and push to main: ruff, mypy, pytest with a 60 % coverage gate, the provided smoke tests as a separate step, and a Docker image build.
- **Multi-stage Dockerfile.** Described in §7; compressed content 86 MB.

9. Limitations & Future Work

- **Wikipedia recall on long questions.** The opensearch endpoint matches article titles, so a full natural-language question often returns zero Wikipedia sources; the pipeline degrades gracefully and says so, but a query-rewriting step (extracting key terms before the Wikipedia call) would raise recall.
- **No streaming output.** The user waits several seconds in silence during synthesis. Gemini supports streaming; the CLI could print tokens as they arrive.
- **Single-process cache.** Two simultaneous asks for the same new question both fetch live and the last writer wins. Entries are idempotent within their TTL so this is harmless at CLI scale, but a multi-process deployment would need file locking or a database.
- **The deterministic benchmark is not a live benchmark.** It proves the orchestrator adds no overhead, by design; it does not predict production latency. The live numbers in `artefacts/demo_run.json` partially fill that gap.
- **Token counts are estimates.** Exact usage would require provider response metadata that the fixed `ai/` interface does not expose.

The worst failure mode on a busy day is Gemini free-tier saturation: with retries exhausted the ask fails with exit code 3. Configuring the failover chain to a second provider is the designed mitigation.

10. Tools & Acknowledgements

We used Claude (Anthropic) as an AI coding assistant throughout the project, in the way the course brief describes as collaborator rather than ghostwriter: it drafted substantial parts of the SE-layer modules, tests, and documentation against the architecture and policies we specified, and everything it produced was reviewed, run, and verified live by the team before being committed. All measured numbers in this report (benchmark, coverage, timings, costs, image size) come from runs we executed ourselves on the submission machine, and the committed artefacts reproduce them. The provided `ai/` package and the course LaTeX templates are the other key external inputs. Per-member ownership is listed in the contribution statement.

References

- [1] Google, Gemini API documentation, <https://ai.google.dev/gemini-api/docs>.

- [2] Tavily search API documentation, <https://docs.tavily.com/>.
- [3] Tenacity documentation, <https://tenacity.readthedocs.io/>.
- [4] pydantic-settings documentation, https://docs.pydantic.dev/latest/concepts/pydantic_settings/.
- [5] Python `asyncio.timeout` (3.11+), <https://docs.python.org/3/library/asyncio-task.html#asyncio.timeout>.
- [6] Wikimedia User-Agent policy, https://meta.wikimedia.org/wiki/User-Agent_policy.
- [7] Course brief, `SOFTWARE_PROJECT.pdf` (AI-ENG-110, Spring 2026).

A. Appendix — Reproducing the benchmark

```
git clone https://github.com/mrorxan/research-assistant
cd research-assistant
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
python scripts/bench.py --N 5 --max-parallel 3
```

No API keys are needed for the benchmark; the fetchers are deterministic stubs. Expected output within a few percent on any modern machine:

```
{
  "n_iterations": 5,
  "sleep_per_source_s": 0.5,
  "max_parallel": 3,
  "sequential_s": 7.589,
  "concurrent_s": 2.535,
  "speedup": 2.99
}
```

For the live path: fill `.env` from `.env.example` with a Gemini and a Tavily key, then `python scripts/demo.py` answers the five sample questions and writes `artefacts/demo_run.json`.

End of report — thank you for reading.